

Modern AI. Course Architecture

Curriculum, projects and course logistics

Author: Alexey Ryabykin • **Course:** Modern AI • **Year:** 2026-2027 • **Release:** v0.0.1

What Is This Course About?

We'll explore how modern AI models work and how to adapt them to a task.

What we'll study

- Model components and the mathematics behind them.
- Training, fine-tuning and ways to reduce compute and memory use.
- Evaluation and the infrastructure needed to run experiments.

How we'll work

- Read papers alongside the code.
- Implement selected components and test them in small experiments.
- Use a project to compare a baseline with our own changes.

I'll bring examples from product work. We'll use them as starting points for experiments and discussion.

Who Is Your Instructor?

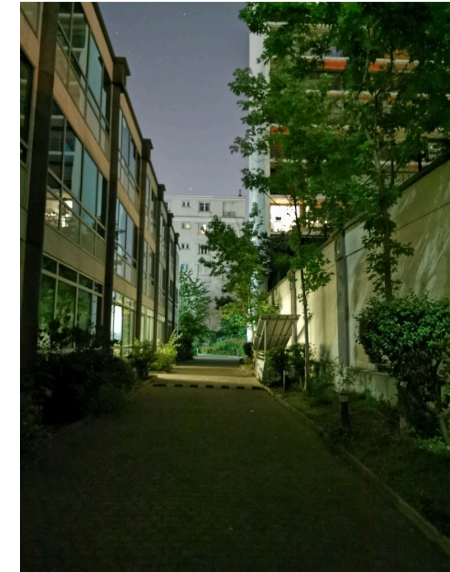
Alexey Ryabykin

Lead Engineer, Huawei Research Center

Project lead for diffusion model deployment on Mate90.

- **Education:** Bachelor's degree from Moscow Aviation Institute. Master's degrees from MIPT and HSE FCS. PhD at Skoltech in progress.
- **Experience:** Contributed to visual model deployment on Mate60, Pura70, Pura X, Mate70, Mate XT, Pura80, Mate80, Pura90 and Mate90. Introduced model optimization tools for pruning, cherry-picking, normalization removal and layer-wise reduction.
- **Projects and research:** `accelerator`: research workflow optimization framework. `qaput`: pruning framework. Diffusion model acceleration.
- **Teaching:** Limited classroom teaching experience: one course as an undergraduate. Currently mentoring three interns at work.
- **Why this course:** This is a good opportunity to deepen my own understanding and share the knowledge I've gained while working on real products.

Huawei Night Mode



FOV Fusion · 4× Zoom



Huawei P30 Pro · Photos: DXOMARK

How Will We Use Telegram?

🎓 Modern Artificial Intelligence

Our course group: class materials, hands-on work with modern AI models, support and project discussions.

📌 00 · Start here

Everything you need to get started: syllabus, schedule, course rules and essential links.

📢 01 · Announcements

Course news, deadlines and schedule updates.

🧩 02 · Prerequisites

Required background and resources to review Python, PyTorch and ML fundamentals.

🔧 03 · Kaggle support

Help with Kaggle, GPUs, environment setup and running notebooks.

📖 04 · Lectures & Seminars

Slides, class recordings, code and questions about lectures and seminars.

🔪 05 · Projects hub

Project descriptions and selection, deliverable requirements and general project questions.

💬 06 · General discussion

Conversation, interesting discoveries and AI discussions beyond the classroom.



Scan to join the course group

Me on Telegram

@addicted-by

How Is the Course Structured?

SYLLABUS OVERVIEW • FROM PRIMITIVES TO FOUNDATION MODELS

COMPREHENSIVE 8-MODULE PROGRESSION

From linear algebra, computational graphs, and loss landscapes to distributed multi-GPU training, large language models, and vision-language synthesis.

M1: Intro & Modalities

Taxonomy of data modalities, generative architectures, Kaggle setup, capstone catalog.

M3: Graphs & Pruning

Autograd internals, reverse-mode autodiff, 1st-order Taylor sensitivity, pruning maps.

M5: Deep Optimization

Spectral optimization (Muon), Newton-Schulz iterations, Lion, SAM, scaling laws.

M7: nanoGPT & SFT

Causal attention, RoPE, RMSNorm, SwiGLU, Qwen2.5 audit, SFT, QLoRA, benchmarks.

M2: Linear Algebra & LoRA

SVD, Eckart-Young theorem, low-rank factorization, LoRA from scratch, PyTorch hooks.

M4: Distillation

First-order optimizers (SGD, AdamW), loss landscapes, local operator distillation.

M6: Distributed & DDP

Data/Tensor parallelism, DDP, FSDP, ZeRO, mixed precision (BF16), Accelerate, W&B.

M8: nanoVLM & Vision

Multimodal fusion, ViT, SigLIP, projection layers, cross-attention, instruction tuning.

Foundations: Algebra, Graphs & Optimization

Modules 01 to 04 • From the current state of AI to computational graphs, automatic matrix differentiation, and optimization.

PART 1 MODULES

- 1** Module 1: Introduction, Modalities & Project Selection
- 2** Module 2: Linear Algebra for Deep Learning & LoRA
- 3** Module 3: Computational Graphs & Taylor Pruning
- 4** Module 4: First-Order Optimization & Local Distillation

Module 1: Introduction, Modalities & Project Selection

WEEKS 01-02 • COURSE LAUNCH

Module 1 Content

- **Course Overview & Logistics:** Grading policy, rules, expectations.
- **Modern AI Landscape:** What AI has already achieved in the world.
- **Data Modalities:** Text, images, audio, video, robotic action.
- **Taxonomy of Architectures:** Autoregression, diffusion, flow matching, masked autoencoders.
- **Multimodal Integration:** Joint embedding spaces and cross-attention alignment.

Seminar 1

- Workspace configuration in **Kaggle**: GPU quotas, secrets, persistent storage.
- Interactive demonstration of baseline notebooks.

Module 1 Homeworks

HW0: BENCHMARK BASELINE MODELS

Run baseline notebooks for 2–3 candidate projects.
Record quality metrics, latency and memory use.

HW1: ARTICLES OVERVIEW

Review the papers behind your project candidates.
Compare their methods, datasets and evaluation, then explain your chosen direction.

Module 2: Linear Algebra for Deep Learning & LoRA

WEEK 03-04 • SUBSPACE GEOMETRY

Module 2 Content

- **Geometric Motivation:** How data manifolds and subspace geometry govern model trainability.
- **Moore-Penrose Pseudoinverse:** Geometric derivation and least-squares connection $X^+ = (X^T X)^{-1} X^T$.
- **Matrix Decompositions:** LU, QR (Gram-Schmidt orthogonalization), Cholesky (covariance and whitening).
- **Singular Value Decomposition (SVD):** Spectral norm, low-rank approximation (Eckart-Young-Mirsky theorem).
- **LoRA (Low-Rank Adaptation):** Weight update parameterization $W = W_0 + \Delta W$, where $\Delta W = \frac{\alpha}{r} B A$.

Seminar 2

- Implementing SVD truncation and LoRA layers from scratch in PyTorch.
- PyTorch **Hooks** mechanism: intercepting activations, weights, and gradients dynamically.
- **Student Presentations:** Capstone concept defenses (HW1).

Module 2 Homeworks

HW2: TORCH / NUMPY/CUPY / SCIPY

Tensor operations and numerical linear algebra across the three library groups.

HW3: FORWARD/BACKWARD/HOOKS

Inspect forward and backward passes. Capture operator inputs, outputs and gradients with hooks. Save the operator setup and initialization for **HW7**.

HW4: ANALYTICAL OPERATOR APPROXIMATION

Apply analytical low-rank approximations to project operators and measure the effect of rank on reconstruction error.

Module 3: Network Foundations, Graphs & Autograd

WEEK 05-06 • GRADIENT MECHANICS

📖 Module 3 Content

- **Computational Graphs:** Static graphs (TorchScript, execution engines) vs Dynamic graphs (PyTorch eager execution).
- **Reverse-Mode Autodiff:** Reverse accumulation, backpropagation, and vector-Jacobian products (VJPs).
- **Gradient as Sensitivity Metric:** First-order Taylor expansion for network pruning:

$$\Delta \mathcal{L} \approx \left| \frac{\partial \mathcal{L}}{\partial w} \cdot w \right|$$

- **Weight vs Activation Sensitivity:** Equivalence proof for linear and convolutional layers.

💻 Seminar 3

- Hands-on **Taylor Pruning:** profiling channel, attention head, and layer sensitivity.
- Building custom backward hooks to accumulate gradient statistics across batches.

🎯 Module 3 Homeworks

HW5: MATRIX DIFFERENTIATION

Derive matrix gradients for the proposed examples.

HW6: TAYLOR IMPORTANCE ESTIMATION

Estimate the importance of layers, channels and attention heads using first-order Taylor scores. Build an importance map for your project model.

$$I(a) = \left| \frac{\partial \mathcal{L}}{\partial a} \odot a \right|$$

Module 4: Optimization Foundations & Local Distillation

WEEK 07 • OPTIMIZATION DYNAMICS

Module 4 Content

- **Momentum Dynamics:** Classical Momentum vs Nesterov Accelerated Gradient (NAG).
- **Adaptive Methods:** RMSprop, Adam, AdamW (decoupled weight decay vs L2 regularization).
- **Loss Landscapes:** Ill-conditioned Hessians, ravines, saddle points, and gradient noise.
- **Local Distillation:** Distilling an expensive operator into a compact surrogate using layerwise alignment:

$$\mathcal{L}_{\text{distill}} = \alpha \mathcal{L}_{\text{MSE}}(y_S, y_T) + \beta (1 - \cos(y_S, y_T))$$

Seminar 4

- Local operator distillation in PyTorch with frozen teacher.

Module 4 Homework

HW7: LOCAL DISTILLATION + INITIALIZATION FROM HW3

1. Select an expensive operator and design a lightweight student.
2. Reuse the forward/backward/hooks setup and initialization from **HW3**.
3. Train the student to reproduce the frozen teacher's outputs.
4. Measure reconstruction quality, latency and memory use.

$$\mathcal{L}_{\text{distill}} = \alpha \mathcal{L}_{\text{MSE}} + \beta \mathcal{L}_{\text{cos}}$$

Scaling: Advanced Optimizers, Infra & LLMs/VLMs

Modules 05 to 08 • Spectral optimization, multi-GPU distributed systems, nanoGPT pretraining & SFT, and multimodal VLM architectures.

PART 2 MODULES

5 Module 5: Deep Optimization, Lion/Muon & Flat Minima

6 Module 6: Infrastructure, Distributed Training & DDP

7 Module 7: nanoGPT — Training LLMs from Scratch & SFT

8 Module 8: nanoVLM — Multimodal Synthesis

Module 5: Deep Optimization, Lion/Muon & Flat Minima

WEEK 08-09 • ADVANCED OPTIMIZATION

📖 Module 5 Content

- **Beyond Standard AdamW:** Why coordinate-wise descent struggles on ill-conditioned matrix manifolds.
- **Muon Optimizer:** Matrix Update Orthogonalization via Newton-Schulz polynomial iterations:

$$O_0 = G/\|G\|_F, \quad O_{k+1} = \frac{1}{2}O_k(3I - O_k^T O_k)$$

- **Lion Optimizer:** Sign-momentum evolution for memory-efficient gradient updates.
- **SAM (Sharpness-Aware Minimization):** Seeking flat minima for superior out-of-distribution generalization:

$$\min_w \max_{\|\epsilon\|_2 \leq \rho} \mathcal{L}(w + \epsilon)$$

- **Scaling Laws:** Compute-optimal training tradeoffs (Chinchilla, Kaplan).

🖥 Seminar 5

- Parameter group splitting: e.g. Muon for 2D weight matrices, AdamW for embeddings and norm layers.

🎯 Module 5 Homework

HW8: ADVANCED LOCAL DISTILLATION + IMPORTANCE ESTIMATIONS FROM HW6

1. Extend the local distillation pipeline from **HW7**.
2. Use the importance estimates from **HW6** to select operators or weight the feature loss.
3. Compare optimization choices, including Muon or Lion.
4. Evaluate convergence and the tradeoff between quality, latency and memory.

Module 6: Infrastructure, Distributed Training & DDP

WEEK 10 • PARALLEL & HIGH-THROUGHPUT COMPUTE

Module 6 Content

- **Parallelism Taxonomy:** Data Parallel (DP), Tensor Parallel (Megatron-LM), Pipeline Parallel, Sequence Parallel.
- **Distributed Systems:** DDP (DistributedDataParallel), ZeRO-1/2/3, and PyTorch FSDP (Fully Sharded Data Parallel).
- **GPU Memory Anatomy:** Model weights, gradients, optimizer states, activations. Offloading & activation checkpointing.
- **Mixed Precision:** FP16, BF16, FP8 dynamic loss scaling.
- **High-Throughput DataLoader:** Prefetching, `num_workers`, pinned memory, online augmentation.
- **Reproducibility:** DVC, Git-LFS, MLflow, Weights & Biases.

Seminar 6

- Distributed launch configuration via **Hugging Face Accelerate**.
- Tracking systems integration: **MLflow, Weights & Biases, Python Fire**.

Module 6 Homework

HW9: INFRASTRUCTURE HW

1. Configure reproducible training and evaluation runs.
2. Set up distributed launch with `accelerate launch` or `torchrun` when multiple GPUs are available.
3. Compare runtime and memory with a single-GPU baseline.
4. Record configs, checkpoints and experiment metrics.

Module 7: nanoGPT/Qwen2.5 — Training LLMs from Scratch & SFT

WEEK 11 • LARGE LANGUAGE ARCHITECTURES

Module 7 Content

- **nanoGPT from Scratch:** BPE tokenization, Causal Self-Attention, KV-cache, Weight Tying.
- **Modern LLM Architecture:** RoPE (Rotary Position Embeddings), RMSNorm, QK-RMSNorm, SwiGLU, GQA (Grouped-Query Attention).
- **Architectural Audit:** In-depth code walkthrough of **Qwen2.5-1.5B**.
- **Pretraining vs SFT:** Next-token loss, ChatML templating, masking user prompt tokens.
- **PEFT (LoRA & QLoRA):** NormalFloat4 (NF4) quantization, double quantization, paged optimizers.
- **LLM Benchmarking:** GSM8K, Math500, MMLU, pass@k, contamination detection.

Seminar 7

- Capstone training scripts: configuring train/eval loops, profiling token throughput and memory.

Final Capstone Project

HW10: PROJECT

1. Train or adapt the model for the chosen task.
2. Compare the final result with the **HW0** baseline.
3. Prepare a reproducible repository, results, demo and final presentation.

Module 8: nanoVLM — Multimodal Synthesis

WEEK 12 • MULTIMODAL FRONTIER

Module 8 Content

- **VLM Architecture:** Unifying visual and language representation manifolds.
- **Core Components:** Visual backbone (ViT, SigLIP), cross-modal projectors (MLP Projector vs Resampler), LLM decoder.
- **Fusion Mechanisms:** Prefix tokens, Cross-Attention, contrastive CLIP/SigLIP objectives.
- **Data & Training:** Image-text instruction pairs, dynamic patch slicing (AnyRes).

Seminar 8

- Empirical comparison of multimodal fusion strategies: convergence rates, zero-shot quality, and compute budgets.

NO NEW HOMEWORK ASSIGNED

No new homework assignment is issued in Module 8!

All student time and compute budget is dedicated to:

1. Finalizing training runs and ablation studies for **HW10**;
2. Validating metrics on held-out test benchmarks;
3. Preparing the final project slide deck, demo, and technical defense.

Curated Capstone Projects Catalog (1/2)

SELECT YOUR SEMESTER TRACK · CLICK A CARD TO EXPLORE

01. DreamBooth Personalization • SOTA / Baseline: FLUX.2 / SDXL / Stable Diffusion 1.5. • Dataset: Custom dataset (5–10 reference photos). • Metrics: DINO-I, CLIP-I, CLIP-T, FID. • Focus: Fine-tuning diffusion models for rare personalized objects.	02. Textual Inversion • SOTA / Baseline: FLUX.2 / SDXL / SD 1.5. • Dataset: 5–10 subject photos, LAION subset. • Metrics: DINO-I, CLIP-I, CLIP-T, GenEval. • Focus: Learning pseudo-tokens in embedding space without changing weights.	03. YOLOv10: Weapon Detection • SOTA / Baseline: YOLOv10S / YOLOv10M. • Dataset: WeaponDetection / Action Recognition Dataset. • Metrics: mAP@50:95, Miss Rate, False Positive Rate, Latency. • Focus: Real-time weapon detection in video streams with automated censorship blur.
04. LoRA Qwen2.5: SQL Generation • SOTA / Baseline: Claude Fable / Qwen2.5-Coder-1.5B-Instruct. • Dataset: <code>b-mc2/sql-create-context</code> + custom SQLite DB. • Metrics: Execution Accuracy (EX), Exact Match (EM), Valid SQL. • Focus: Generating exact SQL queries for complex database schemas.	05. LoRA Qwen2.5: Structured JSON • SOTA / Baseline: Claude Fable / Qwen2.5-Coder-1.5B-Instruct. • Dataset: SchemaBench / JSONSchemaBench (10k schemas). • Metrics: Schema Adherence Accuracy, Valid JSON, Degradation. • Focus: Strict schema adherence without hallucinations.	06. Audio-to-Image Adapter • SOTA / Baseline: FLUX.2 / Stable Diffusion 1.5. • Dataset: MUIImage (9,966 music-image pairs, 27.7h audio). • Metrics: CLIP-A (audio-visual alignment), CLIP-I, FID. • Focus: Conditioning diffusion image synthesis with audio features.

Curated Capstone Projects Catalog (2/2)

SELECT YOUR SEMESTER TRACK · CLICK A CARD TO EXPLORE

07. Object Removal & Inpainting

- **SOTA / Baseline:** FLUX.2 / SDXL / SD 1.5.
- **Dataset:** RemovalBench (paired input/ground-truth frames).
- **Metrics:** FID, LPIPS, GenEval, CLIP-T.
- **Focus:** Consistent object removal and background texture reconstruction.

08. Lyrics Transcription

- **SOTA / Baseline:** ElevenLabs Scribe v2 / Whisper Large-v3-Turbo.
- **Dataset:** M4Singer (29.7 hours of singing vocals, 419 songs).
- **Metrics:** WER, CER, Punctuation F1, Word Timestamps.
- **Focus:** Accurate word-level lyrics transcription with precise timestamp alignment.

09. Hair Unsharpen Enhancement

- **SOTA / Baseline:** SAM 2.1 / SAM 1.
- **Dataset:** LaPa Dataset (Face parsing benchmark).
- **Metrics:** IoU, Dice/F1, Boundary F1, visual sharpness.
- **Focus:** Hair segmentation and frequency-domain unsharp masking:
 $I' = I + \alpha \cdot (I - \text{Blur}(I)).$

10. Portrait Bokeh Depth Blur

- **SOTA / Baseline:** SAM 2.1 / SAM 1.
- **Dataset:** P3M-10k (Portrait matting benchmark).
- **Metrics:** IoU, Boundary F1, Precision/Recall.
- **Focus:** Subject extraction and simulated optical bokeh:
 $I' = M \cdot I + (1 - M) \cdot \text{Blur}(I).$

HOW TO CHOOSE YOUR PROJECT?

1. Assess which modality excites you most (Vision, NLP/Code, Audio, Multimodal).
2. In **Seminar 1**, we will run baseline notebooks for all 10 project tracks.
3. In **HW0**, benchmark 2–3 options on Kaggle.
4. In **HW1**, formally defend your chosen direction.

17

What Are the Homework Assignments?

HW	Topic
HW0	Benchmark Baseline Models
HW1	Articles overview
HW2	torch numpy/cupy scipy
HW3	forward/backward/hooks
HW4	analytical operator approximation
HW5	Matrix differentiation

HW	Topic
HW6	Taylor Importance estimation
HW7	Local distillation + Initialization from HW3
HW8	Advanced local distillation + Importance estimations from HW6
HW9	Infrastructure HW
HW10	Project

What I Hope You'll Get Out of the Course

I hope you'll leave with:

- A better feel for how model code relates to the ideas in a paper.
- Experience running experiments and checking whether a change helps.
- Some practice with debugging, memory limits and training runs.
- A project you understand, with a few ideas for what to try next.

For me, this is also a chance to revisit what I've learned and learn from your questions.